

Tóm tắt nội dung: LLMRAG

Ngày 27 tháng 10 năm 2025

1 Building Agentic RAG with Llamaindex

1.1 Mục tiêu khóa học

Khóa học hướng dẫn cách xây dựng hệ thống **Agentic RAG** bằng thư viện **LlamaIndex**: kết hợp *retrieval* (tìm kiếm tài liệu) với khả năng hành xử như một agent (chọn công cụ, thực hiện reasoning đa bước, xử lý nhiều tài liệu).

1.2 Introduction

- Định nghĩa "Agentic RAG": RAG + khả năng agent (tool-calling, multi-step reasoning).
- Tổng quan về kiến trúc: retrieval → context assembly → LLM reasoning → tool invocation.
- Use-cases: truy vấn phức tạp trên nhiều tài liệu, automation workflows.

1.3 Router Query Engine

- Mô tả: router quyết định engine nào phù hợp với công việc đó (ví dụ: Q&A vs Summary).
- Thiết kế: nhận query, chạy classifier, dispatch đến engine.
- Lợi ích: giảm chi phí, tăng độ chính xác bởi mỗi engine có chuyên môn riêng.

1.4 Tool Calling

- Agent có thể gọi "tools" với index do model sinh ra.
- Cần định nghĩa interface cho tools: tên, mô tả, schema tham số, kết quả trả về.
- Xử lý lỗi: validate args, fallback strategies, timeouts.

1.5 Building an Agent Reasoning Loop

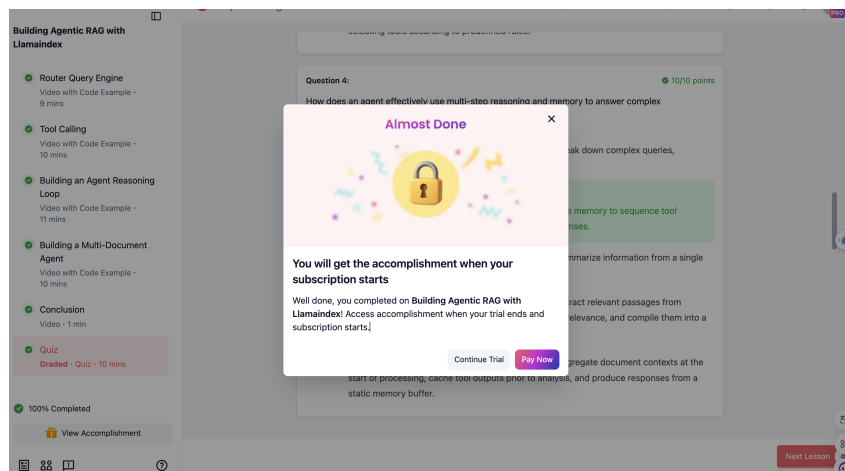
- Vòng lặp reasoning đa bước: think $\rightarrow$ choose tool $\rightarrow$ execute $\rightarrow$ observe $\rightarrow$ think (lặp lại).
- Ghi nhớ trạng thái trung gian (memory) giữa các bước; track tool outputs.
- Kỹ thuật: step-by-step prompts, constrained action spaces, verification steps.

1.6 Building a Multi-Document Agent

- Truy vấn cần tổng hợp thông tin từ nhiều tài liệu.
- Chiến lược retrieval: chunking, ranking theo relevance, cross-document linking.
- Aggregation: merge answers, detect contradictions, provenance tracking.

1.7 Conclusion

- Tóm tắt best practices: select relevant tools early, build granular reasoning traces, manage context size, test with edge cases.



Hình 1: Hoàn thành khoá học Building Agentic RAG with Llamaindex

2 Building and Evaluating Advanced RAG Applications

2.1 Introduction

- Giới thiệu về Retrieval-Augmented Generation (RAG): kết hợp LLM + dữ liệu bên ngoài nhằm tăng tính chính xác và giảm hallucination.
- Mục tiêu: nâng cấp pipeline RAG từ cơ bản lên nâng cao.
- Nội dung:
 - Pipeline RAG nâng cao
 - Bộ chỉ số đánh giá RAG Triad
 - Kỹ thuật cải tiến Retrieval: Sentence-window và Auto-merging
- Lưu ý: cần có kiến thức Python và RAG cơ bản trước.

2.2 Advanced RAG Pipeline

- Các thành phần chính của pipeline:
 1. Ingestion & Document processing
 2. Chunking & Embeddings
 3. Retrieval
 4. Generation
 5. Evaluation & Experiment tracking
- Mục tiêu: cải thiện hiệu suất retrieval và chất lượng câu trả lời.
- Cài đặt thực tế: notebook code mẫu minh họa cách build và thử nghiệm RAG.
- Lưu ý: theo dõi hiệu quả từng giai đoạn để tối ưu dần.

2.3 RAG Triad of Metrics

- Ba tiêu chí đánh giá quan trọng:
 - **Context Relevance**: Ngữ cảnh được truy xuất có đúng với câu hỏi không?
 - **Groundedness**: Câu trả lời có được chứng minh từ ngữ cảnh không?
 - **Answer Relevance**: Câu trả lời có tập trung đúng yêu cầu người dùng không?
- Đánh giá RAG không chỉ là kiểm tra đúng/sai mà phải kiểm tra nguồn tham chiếu.
- Cần có hệ thống đánh giá vòng lặp để tối ưu hóa liên tục.

2.4 Sentence-Window Retrieval

- **Mục tiêu:** Sentence-Window Retrieval nhằm tối ưu hóa RAG, cân bằng giữa độ chính xác tìm kiếm và sự đầy đủ ngữ cảnh.
- **Cơ chế hoạt động (Hai giai đoạn):**
 1. **Giai đoạn 1: Truy xuất Chính xác (Sentence Retrieval)**
 - **Chunking:** Tài liệu được chia thành các đơn vị câu đơn lẻ.
 - **Indexing:** Mỗi câu được nhúng (embedding) và lưu trữ.
 - **Tìm kiếm:** Hệ thống truy vấn để tìm ra K câu có độ liên quan cao nhất (mảnh truy xuất).
 - → Lợi ích: Độ chính xác ngữ nghĩa cao.
 2. **Giai đoạn 2: Mở Cửa sổ Ngữ cảnh (Windowing Synthesis)**
 - **Mở rộng:** Với mỗi câu được truy xuất, hệ thống mở một "cửa sổ" bằng cách lấy thêm N câu trước và N câu sau từ tài liệu gốc.
 - **Tổng hợp:** Các đoạn văn bản "cửa sổ" đã được mở rộng này (ngữ cảnh hoàn chỉnh) được gửi đến LLM.
 - → Lợi ích: Cung cấp bối cảnh đầy đủ, giảm ảo giác (hallucination) do thiếu thông tin.

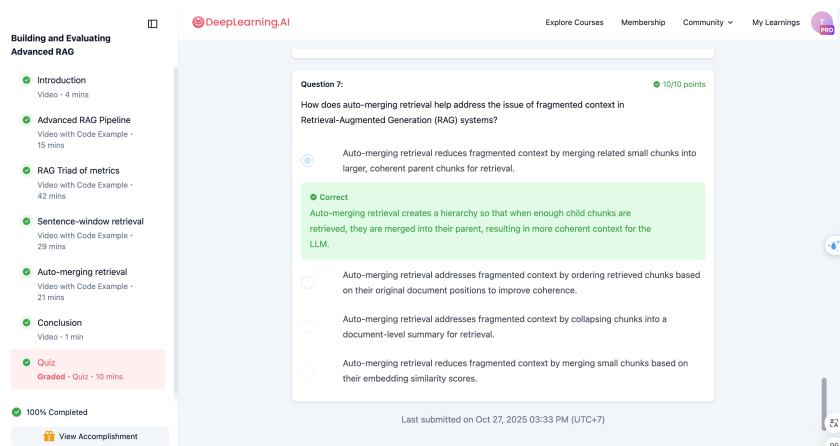
2.5 Auto-merging Retrieval

- **Giới thiệu:**
 - Là một kỹ thuật nâng cao trong các hệ thống RAG
 - Mục đích là khắc phục sự đánh đổi giữa việc tìm kiếm chính xác (cần mảnh nhỏ) và cung cấp ngữ cảnh đầy đủ cho LLM (cần mảnh lớn).
- **Cơ chế hoạt động:** Kỹ thuật này sử dụng một cấu trúc phân cấp hai cấp độ khi chia tài liệu (chunking)
 - **Phân cấp Chunking (Mảnh):**
 - * Mảnh Con (Child Nodes): Các mảnh văn bản rất nhỏ (ví dụ: 1-3 câu).
-> Được dùng để truy xuất (Tìm kiếm), nhờ vào độ chính xác cao của vector nhúng.
 - * Mảnh Cha (Parent Nodes): Các mảnh văn bản lớn hơn (ví dụ: một đoạn văn).
-> Được dùng để tổng hợp (Trả lời), cung cấp ngữ cảnh toàn diện cho LLM.
 - **Quy trình "Tự động Hợp nhất":**
 1. Truy xuất chính xác: Hệ thống tìm kiếm trong cơ sở dữ liệu vector chỉ bằng cách sử dụng các mảnh con.
 2. Xác định Mảnh Cha: Với mỗi Mảnh Con liên quan được tìm thấy, hệ thống xác định mảnh cha tương ứng.
 3. Hợp nhất (Merging): Nếu nhiều Mảnh Con được truy xuất cùng trở đến cùng một Mảnh Cha, chúng sẽ được hợp nhất. Hệ thống chỉ trả về Mảnh Cha đó một lần duy nhất.

4. Tổng hợp (Synthesis): Mảnh Cha lớn hơn (đã được hợp nhất) sẽ được gửi đến LLM để tạo ra câu trả lời cuối cùng.

2.6 Conclusion

- Tóm tắt kiến thức:
 - Pipeline nâng cao
 - Bộ metrics chuẩn cho RAG
 - Hai kỹ thuật tối ưu retrieval



Hình 2: Hoàn thành khoá học Building and Evaluating Advanced RAG